

LAPORAN TUGAS 5

Pemrograman JavaScript - Event Listener, Destructuring, Spread, dan Rest Parameter

APLIKASI MANAJEMEN PRODUK TOKO ONLINE

Event Listener, Destructuring, Spread Operator, dan Rest Parameter

DATA DIRI

Nama	:	Latif Wibowo
-------------	---	--------------

NIM	:	0110224065
------------	---	------------

Program Studi	:	Teknik Informatika
----------------------	---	--------------------

Institusi	:	STT Terpadu Nurul Fikri
------------------	---	-------------------------

1. Deskripsi Tugas

Sebuah toko online ingin mengembangkan fitur manajemen produk berbasis JavaScript. Aplikasi menggunakan **Event Listener**, **Destructuring**, **Spread Operator**, dan **Rest Parameter** untuk meningkatkan efisiensi dan fleksibilitas kode.

Fitur yang harus ada:

- List data produk awal minimal 5 produk.
- Menambahkan produk.
- Menghapus produk.
- Menampilkan semua produk.

Penerapan konsep pada program

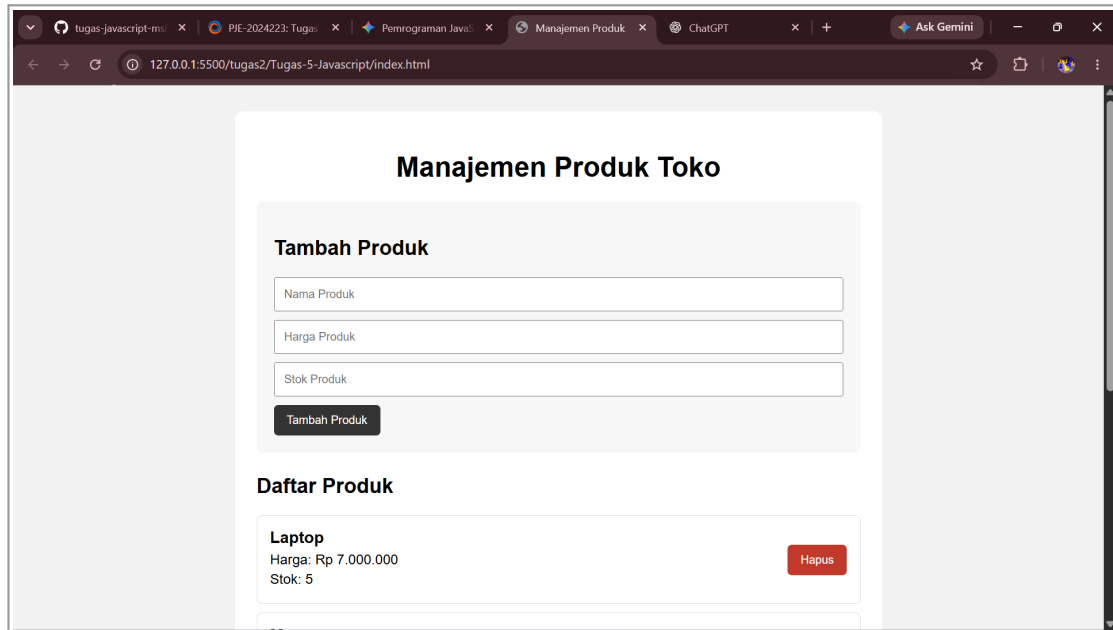
Konsep	Lokasi di script.js	Fungsi
Rest Parameter	buatProduk(...produkBaru)	Mengumpulkan argumen menjadi array
Destructuring	tampilkanProduk() dan buatProduk()	Mengambil properti/elemen ke variabel
Spread Operator	tambahProduk()	Membuat array baru: [...produk, produkBaru]
Event Listener	addEventListener pada #btnTambah	Menjalankan tambahProduk saat tombol diklik

Struktur File Proyek

File	Fungsi
index.html	Form tambah produk dan wadah daftar produk
script.js	Data produk, fungsi tambah, hapus, tampil, dan event listener
style.css	Tampilan halaman, form, dan kartu produk

2. Hasil Program

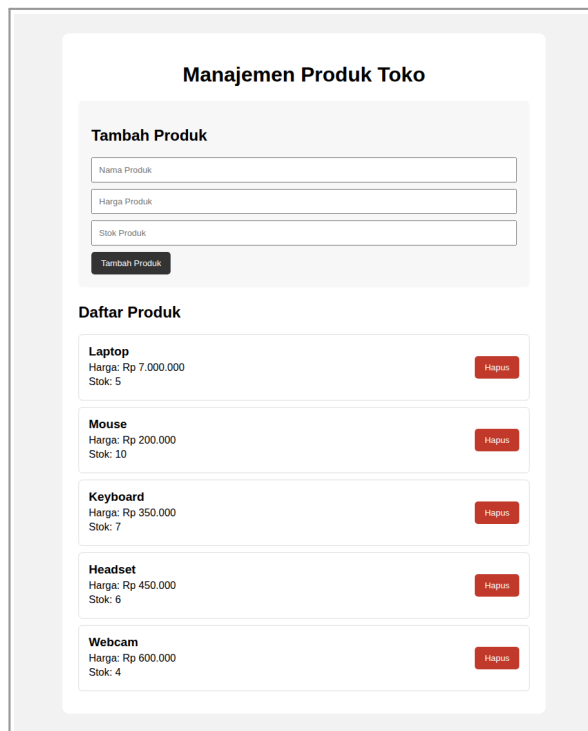
2.1 Tampilan Halaman di Browser



Gambar 1. Tampilan halaman index.html di browser (alamat 127.0.0.1:5500)

Halaman menampilkan judul **Manajemen Produk Toko**, form **Tambah Produk** (nama, harga, stok), serta **Daftar Produk** yang setiap produknya dilengkapi tombol Hapus berwarna merah. Produk pertama yang tampil adalah Laptop (Rp 7.000.000, stok 5).

2.2 Seluruh Daftar Produk Awal (5 Produk)



Gambar 2. Halaman dimuat penuh: lima produk awal tampil dari array produk

Kelima data awal tampil sesuai array: Laptop, Mouse (Rp 200.000, stok 10), Keyboard (Rp 350.000, stok 7), Headset (Rp 450.000, stok 6), dan Webcam (Rp 600.000, stok 4).

2.3 Menambahkan Produk (Event Listener dan Spread Operator)

Manajemen Produk Toko

Tambah Produk

Daftar Produk

Gambar 3. Form diisi data produk baru: Tablet, harga 7000000, stok 3

Manajemen Produk Toko

Tambah Produk

Daftar Produk

Laptop Harga: Rp 7.000.000 Stok: 5	Hapus
Mouse Harga: Rp 200.000 Stok: 10	Hapus
Keyboard Harga: Rp 350.000 Stok: 7	Hapus
Headset Harga: Rp 450.000 Stok: 6	Hapus
Webcam Harga: Rp 600.000 Stok: 4	Hapus
Tablet Harga: Rp 7.000.000 Stok: 3	Hapus

Gambar 4. Setelah tombol Tambah Produk diklik, Tablet muncul di akhir daftar (6 produk) Klik pada tombol memicu event listener yang menjalankan **tambahProduk()**. Produk Tablet ditambahkan lewat spread operator, daftar diperbarui otomatis, dan kolom input kembali kosong. Jika ada kolom yang dibiarkan kosong, muncul alert "Semua data produk harus diisi!" dan produk tidak ditambahkan.

2.4 Menghapus Produk

Manajemen Produk Toko

Tambah Produk

Tambah Produk

Daftar Produk

Laptop
Harga: Rp 7.000.000
Stok: 5

Hapus

Keyboard
Harga: Rp 350.000
Stok: 7

Hapus

Headset
Harga: Rp 450.000
Stok: 6

Hapus

Webcam
Harga: Rp 600.000
Stok: 4

Hapus

Tablet
Harga: Rp 7.000.000
Stok: 3

Hapus

Gambar 5. Setelah tombol Hapus pada produk Mouse diklik, produk tersebut hilang dari daftar (5 produk)

Tombol **Hapus** memanggil **hapusProduk(id)**. Produk Mouse (ID 2) hilang dari daftar, sehingga tersisa Laptop, Keyboard, Headset, Webcam, dan Tablet.

Catatan: Gambar 2 sampai 5 dihasilkan dengan membuka `index.html` pada browser (Chromium) dari source code tugas, lalu menjalankan langkah isi form, klik Tambah, dan klik Hapus.

3. Source Code Penting

Berikut potongan source code yang menjadi inti tugas, disertai penjelasan masing-masing.

3.1 Data Produk Awal (5 Produk)

```
1 // Data produk awal
2 let produk = [
3   {
4     id: 1,
5     nama: "Laptop",
6     harga: 7000000,
7     stok: 5
8   },
9   {
10    id: 2,
11    nama: "Mouse",
12    harga: 200000,
13    stok: 10
14  },
15  {
16    id: 3,
17    nama: "Keyboard",
18    harga: 350000,
19    stok: 7
20  },
21  {
22    id: 4,
23    nama: "Headset",
24    harga: 450000,
25    stok: 6
26  },
27  {
28    id: 5,
29    nama: "Webcam",
30    harga: 600000,
31    stok: 4
32  },
33 ];
```

Gambar 6. script.js (baris 1-33)

Array **produk** berisi lima data awal sesuai ketentuan minimal 5 produk: Laptop, Mouse, Keyboard, Headset, dan Webcam. Setiap produk berupa objek dengan properti **id**, **nama**, **harga**, dan **stok**. Array dideklarasikan dengan **let** karena isinya akan diganti saat produk ditambah atau dihapus.

3.2 Rest Parameter pada buatProduk()

```
35 // Rest parameter
36 function buatProduk(...produkBaru) {
37   const [id, nama, harga, stok] = produkBaru;
38
39   return {
40     id: id,
41     nama: nama,
42     harga: harga,
43     stok: stok
44   };
45 }
```

Gambar 7. script.js (baris 35-45)

Fungsi **buatProduk(...produkBaru)** memakai **rest parameter** (...) untuk mengumpulkan semua argumen menjadi satu array. Array tersebut kemudian di-*destructuring* menjadi **id**, **nama**, **harga**, dan **stok**, lalu dikembalikan sebagai objek produk baru.

3.3 Destructuring pada tampilkanProduk()

```
47 // Menampilkan semua produk
48 function tampilkanProduk() {
49
50     const listProduk = document.getElementById("listProduk");
51
52     listProduk.innerHTML = "";
53
54     produk.forEach((item) => {
55         // Destructuring
56         const { id, nama, harga, stok } = item;
57
58         const div = document.createElement("div");
59         div.classList.add("produk");
60
61         div.innerHTML = `
62             <div>
63                 <h3>${nama}</h3>
64                 <p>Harga: Rp ${harga.toLocaleString("id-ID")}</p>
65                 <p>Stok: ${stok}</p>
66             </div>
67
68             <button class="btn-hapus" onclick="hapusProduk(${id})" >
69                 Hapus
70             </button>
71         `;
72         listProduk.appendChild(div);
73     });
74 }
75 }
```

Gambar 8. script.js (baris 47-75)

Fungsi ini mengosongkan wadah #listProduk lalu menelusuri array dengan forEach. Pada tiap produk, **destructuring** `const { id, nama, harga, stok } = item` dipakai untuk mengambil properti objek langsung ke variabel. Setiap produk dirender sebagai kartu berisi nama, harga (diformat rupiah), stok, dan tombol **Hapus** yang memanggil `hapusProduk(id)`.

3.4 Spread Operator pada tambahProduk()

```
77 // Menambahkan produk
78 function tambahProduk() {
79     const nama = document.getElementById("namaProduk").value;
80     const harga = Number(document.getElementById("hargaProduk").value);
81     const stok = Number(document.getElementById("stokProduk").value);
82
83     if (nama === "" || harga === 0 || stok === 0) {
84         alert("Semua data produk harus diisi!");
85         return;
86     }
87
88     const idBaru = produk.length + 1;
89
90     const produkBaru = buatProduk(
91         idBaru,
92         nama,
93         harga,
94         stok
95     );
96
97     // Spread operator
98     produk = [...produk, produkBaru];
99
100    tampilkanProduk();
101
102    document.getElementById("namaProduk").value = "";
103    document.getElementById("hargaProduk").value = "";
104    document.getElementById("stokProduk").value = "";
105}
```

Gambar 9. script.js (baris 77-105)

Fungsi mengambil nilai dari tiga input form dan memvalidasinya; jika ada yang kosong, muncul alert "Semua data produk harus diisi!". ID baru dibuat dari `produk.length + 1` dan objek dibentuk lewat `buatProduk()`. Produk ditambahkan dengan **spread operator** `produk = [...produk, produkBaru]`, yaitu membuat array baru berisi semua produk lama ditambah produk baru. Setelah daftar ditampilkan ulang, kolom input dikosongkan.

3.5 hapusProduk() dan Event Listener

```
107// Menghapus produk
108function hapusProduk(id) {
109
110    produk = produk.filter((item) => item.id !== id);
111
112    tampilkanProduk();
113}
114
115// Event listener
116document.getElementById("btnTambah").addEventListener("click", tambahProduk);
117
118// Menampilkan produk saat halaman dimuat
119tampilkanProduk();
```

Gambar 10. script.js (baris 107-119)

Fungsi `hapusProduk(id)` menghapus produk dengan `filter()`, yaitu menyisakan semua produk yang ID-nya tidak sama dengan ID yang dihapus. **Event listener** `addEventListener("click", tambahProduk)` dipasang pada tombol Tambah Produk, dan `tampilkanProduk()` dipanggil di akhir agar daftar langsung tampil saat halaman dimuat.

3.6 Struktur Halaman Web

```
11     <div class="container">
12         <h1>Manajemen Produk Toko</h1>
13
14         <div class="form-produk">
15             <h2>Tambah Produk</h2>
16
17             <input type="text" id="namaProduk" placeholder="Nama Produk">
18             <input type="number" id="hargaProduk" placeholder="Harga Produk">
19             <input type="number" id="stokProduk" placeholder="Stok Produk">
20
21             <button id="btnTambah">Tambah Produk</button>
22         </div>
23
24         <div class="daftar-produk">
25             <h2>Daftar Produk</h2>
26
27             <div id="listProduk"></div>
28         </div>
29     </div>
```

Gambar 11. index.html (baris 11-29)

HTML menyediakan form tambah produk (tiga input dan tombol #btnTambah) serta wadah kosong #listProduk yang diisi oleh JavaScript. Id pada elemen-elemen ini dipakai script untuk mengambil nilai input dan memasang event listener.

3.7 Styling Kartu Produk dan Tombol Hapus

```
47 .produk {
48     display: flex;
49     justify-content: space-between;
50     align-items: center;
51     border: 1px solid #ddd;
52     padding: 15px;
53     margin-bottom: 10px;
54     border-radius: 6px;
55 }
56
57 .produk h3 {
58     margin: 0 0 5px 0;
59 }
60
61 .produk p {
62     margin: 4px 0;
63 }
64
65 .btn-hapus {
66     background-color: #c0392b;
67 }
```

Gambar 12. style.css (baris 47-67)

Kelas .produk memakai **flexbox** (justify-content: space-between) sehingga informasi produk berada di kiri dan tombol Hapus di kanan. Kelas .btn-hapus memberi warna merah pada tombol hapus agar mudah dikenali.

4. Kesimpulan

Aplikasi manajemen produk berhasil dibuat sesuai petunjuk tugas. Program memiliki lima produk awal, serta fitur menambah, menghapus, dan menampilkan produk. Keempat konsep yang diminta diterapkan: **rest parameter** pada `buatProduk()`, **destructuring** pada `tampilkanProduk()`, **spread operator** pada `tambahProduk()`, dan **event listener** pada tombol Tambah Produk.